

Value Iteration and Policy Iteration for Solving the Bellman Equation

Value iteration

- Recall the Bellman equation for infinite horizon discounted RL

$$V^*(x) = \max_a E [r(x, a) + \alpha V^*(x')] .$$

Value iteration

- Recall the Bellman equation for infinite horizon discounted RL

$$V^*(x) = \max_a E [r(x, a) + \alpha V^*(x')] .$$

- Suppose that both the state space $\mathcal{X}$ and action space $\mathcal{A}$ are finite, $P_{x,x'}$ and $\bar{r}(x, a) = E[r(x, a)]$ are known (model-based).

Value iteration

- Recall the Bellman equation for infinite horizon discounted RL

$$V^*(x) = \max_a E [r(x, a) + \alpha V^*(x')] .$$

- Suppose that both the state space $\mathcal{X}$ and action space $\mathcal{A}$ are finite, $P_{x,x'}$ and $\bar{r}(x, a) = E[r(x, a)]$ are known (model-based).
- Starting from V_0 , we can iteratively compute

$$V_{k+1} \leftarrow \max_a \left(\bar{r}(x, a) + \alpha \sum_{x'} P_{x,x'}(a) V_k(x') \right) .$$

Value iteration

Starting from V_0 , we can iteratively compute

$$V_{k+1} \leftarrow \max_a \left(\bar{r}(x, a) + \alpha \sum_{x'} P_{x,x'}(a) V_k(x') \right).$$

$$V_k \rightarrow V^*$$

After finding V^* , we can find the optimal policy by solving

$$\arg \max_a \left(\bar{r}(x, a) + \alpha \sum_{x'} P_{x,x'}(a) V^*(x') \right).$$

Value iteration: Grid world example

- Two actions: clockwise (c) or counter-clockwise (cc).

1 +1	2	3
8 -1		4
7	6	5

Value iteration: Grid world example

- Two actions: clockwise (c) or counter-clockwise (cc).
- The agent follows the action with probability 0.8 and moves to the opposite direction with probability 0.2.

1 +1	2	3
8 -1		4
7	6	5

Value iteration: Grid world example

- Two terminal states with nonzero rewards: +1 in state 1, -1 in state 8. So for example, $r(2, cc) = 1$ with probability 0.8 and $r(2, cc) = 0$ with probability 0.2.

1 +1	2	3
8 -1		4
7	6	5

Value iteration: Grid world example

$V_k(1) = V_k(8) = 0$ for all k . Define

$$\mathbf{v}_k = \begin{pmatrix} V_k(2) \\ V_k(3) \\ V_k(4) \\ V_k(5) \\ V_k(6) \\ V_k(7) \end{pmatrix} \text{ and assume } V_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

1 +1	2	3
8 -1		4
7	6	5

Value iteration: Grid world example

$$V_1(2) = \max_{a \in \{c, cc\}} E [r(2, a) + 0.9V_0(x')]$$

1 +1	2	3
8 -1		4
7	6	5

Value iteration: Grid world example

$$V_1(2) = \max_{a \in \{c, cc\}} E[r(2, a) + 0.9V_0(x')]$$

$$E[r(2, cc) + 0.9V_0(x')] = 0.8 \times 1 + 0.2 \times 0 = 0.8$$

$$E[r(2, c) + 0.9V_0(x')] = 0.2 \times 1 + 0.8 \times 0 = 0.2$$

1 +1	2	3
8 -1		4
7	6	5

Value iteration: Grid world example

$$\begin{aligned} V_1(2) &= \max_{a \in \{c, cc\}} E [r(2, a) + 0.9V_0(x')] \\ &= \max\{0.8, 0.2\} = 0.8. \end{aligned}$$

1 +1	2	3
8 -1		4
7	6	5

Value iteration: Grid world example

$$V_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \rightarrow V_1 = \begin{pmatrix} 0.8 \\ 0 \\ 0 \\ 0 \\ 0 \\ -0.2 \end{pmatrix} \rightarrow V_2 = \begin{pmatrix} 0.8 \\ 0.576 \\ 0 \\ 0 \\ -0.036 \\ -0.2 \end{pmatrix}$$

1 +1	2	3
8 -1		4
7	6	5

Value iteration: Grid world example

$$V^* = \begin{pmatrix} 0.9444 \\ 0.8023 \\ 0.6796 \\ 0.5661 \\ 0.4269 \\ 0.1074 \end{pmatrix}$$

1 +1	2	3
8 -1		4
7	6	5

Policy iteration

- Choose an initial policy μ , set $k = 0$
- Compute the value function of the policy V_{μ_k} by solving the Bellman equation.

$$V_{\mu_k}(i) = E[r(i, u)] + \alpha \sum_j P_{ij}(u) V_{\mu_k}(j)$$

- Compute a new policy μ_{k+1} as

$$\mu_{k+1}(i) \leftarrow \arg \max_u E[r(i, u)] + \alpha \sum_j P_{ij}(u) V_{\mu_k}(j)$$

- Repeat if $V_{\mu_{k+1}} \neq V_{\mu_k}$

Policy iteration

$V_{\mu_{k+1}} \geq V_{\mu_k}$, i.e. the value function improves at each step before the algorithm terminates.

Policy iteration: Grid world example

$\mu_0(i) = c$, i.e. the initial policy is to move clockwise in any state.

$$V_{\mu_0}(i) = E[r(i, c)] + 0.9 (0.8 V_{\mu_0}(i + 1) + 0.2 V_{\mu_0}(i - 1))$$

$$V_{\mu_0} = \begin{pmatrix} -0.1074 \\ -0.4269 \\ -0.5661 \\ -0.6796 \\ -0.8023 \\ -0.9444 \end{pmatrix}$$

1 +1	2	3
8 -1		4
7	6	5

Policy iteration: Grid world example

$\mu_1(i) = cc$, i.e. the improved policy is to go counter-clockwise (the optimal policy)

1 +1	2	3
8 -1		4
7	6	5

Policy iteration: Grid world example

$\mu_1(i) = cc$, i.e. the improved policy is to go counter-clockwise (the optimal policy)

$$\mu_1(4) = \arg \max_{cc, c} \{ 0.9 \times (0.8 \times -0.5661 + 0.2 \times -0.6796), \\ 0.9 \times (0.2 \times -0.5661 + 0.8 \times -0.6796) \} = cc$$

1 +1	2	3
8 -1		4
7	6	5

Challenges of Applying These Algorithms

Challenges

Model-based algorithms: need to know $\bar{r}(i, u)$ and $P_{ij}(u)$

Challenges of Applying These Algorithms

Challenges

Model-based algorithms: need to know $\bar{r}(i, u)$ and $P_{ij}(u)$

Scalability: only work for finite state space and finite action space

Learning is needed when the system model is unknown.

Q-learning (Watkins '89)

Define Q-function (also called action value function):

$$Q(i, \mathbf{u}) = \bar{r}(i, \mathbf{u}) + \alpha \sum_j P_{ij}(\mathbf{u}) V^*(j).$$

Q-learning (Watkins '89)

Define Q-function (also called action value function):

$$Q(i, u) = \bar{r}(i, u) + \alpha \sum_j P_{ij}(u) V^*(j).$$

Given Q , we can find the optimal policy by taking

$$\max_u Q(i, u)$$

(Note: does not require the model $P_{ij}(u)$)

Q-learning (Watkins '89)

Define Q-function (also called action value function):

$$Q(i, u) = \bar{r}(i, u) + \alpha \sum_j P_{ij}(u) V^*(j).$$

Given Q , we can find the optimal policy by taking

$$\max_u Q(i, u)$$

(Note: does not require the model $P_{ij}(u)$)

Q-learning: A learning algorithm to learn the Q-function.

Q-learning (Watkins '89)

Q-learning (off-policy)

Let $\tilde{Q}$ be the estimate of Q . Given an experience (current state $x_k = i$, current action $a_t = u$, received reward $r(i, u)$, and next state $x_{k+1} = j$), Q-learning updates $\tilde{Q}$ as follows:

$$\begin{aligned}\tilde{Q}(i, u) &\leftarrow (1 - \beta_k) \tilde{Q}(i, u) + \beta_k (r(i, u) + \alpha \max_v \tilde{Q}(j, v)) \\ &= \tilde{Q}(i, u) + \beta_k \left(r(i, u) + \alpha \max_v \tilde{Q}(j, v) - \tilde{Q}(i, u) \right)\end{aligned}$$

$$Q(i, u) = \bar{r}(i, u) + \alpha \sum_j P_{ij}(u) \mathbf{V}^*(j) = \bar{r}(i, u) + \alpha \sum_j P_{ij}(u) \mathbf{\max}_v Q(j, v).$$

Q-Learning: Grid world example

Assume the current estimate is

(2,c) 0.4	(3,c) 0.38	(4,c) 0.14	(5,c) 0.2	(6,c) -0.8	(7,c) -0.9
(2,cc) 0.85	(3,cc) 0.78	(4,cc) 0.34	(5,cc) 0.3	(6,cc) -0.5	(7,cc) -0.34

1 +1	2	3
8 -1		4
7	6	5

Q-Learning: Grid world example

Assume the current estimate is

(2,c)	(3,c)	(4,c)	(5,c)	(6,c)	(7,c)
0.4	0.38	0.14	0.2	-0.8	-0.9
(2,cc)	(3,cc)	(4,cc)	(5,cc)	(6,cc)	(7,cc)
0.85	0.78	0.34	0.3	-0.5	-0.34

Given experience $(x_k = 5, u_k = c, r_k = 0, x_{k+1} = 6)$, Q-learning updates $\tilde{Q}(5, c)$

$$\tilde{Q}(5, c) \leftarrow 0.2 + \beta_k (0 + \alpha \max\{-0.8, -0.5\} - 0.2)$$

$$\tilde{Q}(i, u) \leftarrow \tilde{Q}(i, u) + \beta_k \left(r(i, u) + \alpha \max_v \tilde{Q}(j, v) - \tilde{Q}(i, u) \right)$$

State-action-reward-state-action (SARSA) (Rummery & Niranjan '94)

SARSA (on-policy)

- At step k , with probability $1 - \epsilon_k$, choose action u_k such that

$$u_k \in \arg \max_v \tilde{Q}(x_k, v)$$

and with probability ϵ_k , choose an action u_k uniformly at random. Observe x_{k+1} and $r(x_k, u_k)$.

- With data $(x_{k-1}, u_{k-1}, x_k, u_k)$, update Q such that

$$\tilde{Q}(x_{k-1}, u_{k-1}) \leftarrow (1 - \beta_k) \tilde{Q}(x_{k-1}, u_{k-1}) + \beta_k (r(x_{k-1}, u_{k-1}) + \alpha \tilde{Q}(x_k, u_k))$$

- Choose $\{\epsilon_k\}$ such that $\epsilon_k \rightarrow 0$ as $k \rightarrow \infty$

SARSA: Grid world example

Assume at time k , the estimate is

(2,c) 0.4	(3,c) 0.38	(4,c) 0.14	(5,c) 0.2	(6,c) -0.8	(7,c) -0.9
(2,cc) 0.85	(3,cc) 0.78	(4,cc) 0.34	(5,cc) 0.3	(6,cc) -0.5	(7,cc) -0.34

and assume $x_{k-1} = 4$, $u_{k-1} = cc$, $r_{k-1} = 0$, and $x_k = 5$.

SARSA: Grid world example

Assume at time k , the estimate is

(2,c) 0.4	(3,c) 0.38	(4,c) 0.14	(5,c) 0.2	(6,c) -0.8	(7,c) -0.9
(2,cc) 0.85	(3,cc) 0.78	(4,cc) 0.34	(5,cc) 0.3	(6,cc) -0.5	(7,cc) -0.34

and assume $x_{k-1} = 4$, $u_{k-1} = cc$, $r_{k-1} = 0$, and $x_k = 5$.

SARSA chooses action:

$$u_k = \arg \max_{c, cc} \{ \tilde{Q}(5, c) = 0.2, \tilde{Q}(5, cc) = 0.3 \} = cc.$$

SARSA: Grid world example

Assume at time k , the estimate is

(2,c) 0.4	(3,c) 0.38	(4,c) 0.14	(5,c) 0.2	(6,c) -0.8	(7,c) -0.9
(2,cc) 0.85	(3,cc) 0.78	(4,cc) 0.34	(5,cc) 0.3	(6,cc) -0.5	(7,cc) -0.34

and assume $x_{k-1} = 4$, $u_{k-1} = cc$, $r_{k-1} = 0$, and $x_k = 5$.

SARSA updates $\tilde{Q}(4, cc)$:

$$\tilde{Q}(4, cc) \leftarrow 0.34 + \beta_k (0 + \alpha \times 0.3 - 0.34) .$$

$$\tilde{Q}(x_{k-1}, u_{k-1}) \leftarrow (1 - \beta_k) \tilde{Q}(x_{k-1}, u_{k-1}) + \beta_k (r(x_{k-1}, u_{k-1}) + \alpha \tilde{Q}(x_k, u_k))$$

Off-policy vs. on-policy reinforcement learning

Target policy: The policy to be learned

Behavior policy: The policy used to generate samples

- Q-learning: target policy - optimal policy
behavior policy - any policy under which each action is taken infinitely often
- SARSA: target policy - ϵ -greedy
behavior policy - ϵ -greedy

Reinforcement Learning with Function Approximations

Challenges in solving the Bellman equation:

- Model-based algorithms: need to know $\bar{r}(i, u)$ and $P_{ij}(u)$
→ model free, data-driven methods. Q-Learning, SARSA, etc

Reinforcement Learning with Function Approximations

Challenges in solving the Bellman equation:

- Model-based algorithms: need to know $\bar{r}(i, u)$ and $P_{ij}(u)$
→ model free, data-driven methods. Q-Learning, SARSA, etc
- Scalability or Curse-of-dimensionality: large state and action spaces (the 9×9 Go game has 10^{35} states).
→ function approximation. Linear Function Approximation or Deep Neural Networks

Linear Function Approximation (Feature-Based)

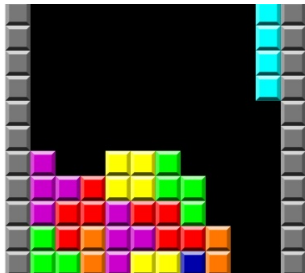
$$Q(i, u) \approx \sum_{m=1}^M \theta_m \phi_m(i, u) = \theta^T \phi(i, u)$$

- $\phi(i, u)$: M -dimensional feature vector for (i, u) and is assumed to be given
- θ : M -dimensional weight vector; common for all (state, action) pairs.

Linear Function Approximation (Feature-Based)

$$Q(i, u) \approx \sum_{m=1}^M \theta_m \phi_m(i, u) = \theta^T \phi(i, u)$$

- $\phi(i, u)$: M -dimensional feature vector for (i, u) and is assumed to be given
- θ : M -dimensional weight vector; common for all (state, action) pairs.



Features: the height of the highest column, differences in height between neighboring columns, holes, and wells

Picture is from <https://en.wikipedia.org/wiki/Tetris>

Linear Function Approximation (Feature-Based)

$$Q(i, u) \approx \sum_{m=1}^M \theta_m \phi_m(i, u) = \theta^T \phi(i, u)$$

- $\phi(i, u)$: M -dimensional feature vector for (i, u) and is assumed to be given
- θ : M -dimensional weight vector; common for all (state, action) pairs.

Learning $Q(i, u)$ ($|\mathcal{S}| \times |\mathcal{A}|$ pairs) is equivalent to learning θ .

- If $M \ll |\mathcal{S}| \times |\mathcal{A}|$, the complexity reduces significantly.
- Learning θ also allows us to generalize when facing new state-action pair.

Q-Learning with Linear Function Approximation

Temporal Difference

$$\delta_k = r(x_k, u_k) + \alpha \max_v \tilde{Q}(x_{k+1}, v) - \tilde{Q}(x_k, u_k)$$

Q-Learning with Linear Function Approximation

Temporal Difference

$$\delta_k = r(x_k, u_k) + \alpha \max_v \tilde{Q}(x_{k+1}, v) - \tilde{Q}(x_k, u_k)$$

Temporal Difference with Linear Function Approximation

$$\delta_k(\tilde{\theta}) = r(x_k, u_k) + \alpha \max_v \tilde{\theta}^T \phi(x_{k+1}, v) - \tilde{\theta}^T \phi(x_k, u_k)$$

Q-Learning with Linear Function Approximation

Temporal Difference

$$\delta_k = r(x_k, u_k) + \alpha \max_v \tilde{Q}(x_{k+1}, v) - \tilde{Q}(x_k, u_k)$$

Q-Learning

$$\tilde{Q}(x_k, u_k) \leftarrow \tilde{Q}(x_k, u_k) + \beta_k \left(r(x_k, u_k) + \alpha \max_v \tilde{Q}(x_{k+1}, v) - \tilde{Q}(x_k, u_k) \right)$$

$$\tilde{Q}(x_k, u_k) \leftarrow \tilde{Q}(x_k, u_k) + \beta_k \delta_k \approx \tilde{Q}(x_k, u_k) - \frac{1}{2} \beta_k \frac{\partial \delta_k^2}{\partial \tilde{Q}(x_k, u_k)}$$

Q-Learning with Linear Function Approximation

Temporal Difference with Linear Function Approximation

$$\delta_k(\tilde{\theta}) = r(x_k, u_k) + \alpha \max_v \tilde{\theta}^T \phi(x_{k+1}, v) - \tilde{\theta}^T \phi(x_k, u_k)$$

Q-Learning with Linear Function Approximation

Temporal Difference with Linear Function Approximation

$$\delta_k(\tilde{\theta}) = r(x_k, u_k) + \alpha \max_v \tilde{\theta}^T \phi(x_{k+1}, v) - \tilde{\theta}^T \phi(x_k, u_k)$$

Q-Learning with Linear Function Approximation

$$\tilde{\theta} \leftarrow \tilde{\theta} - \frac{1}{2} \beta_k \frac{\partial \delta_k^2(\tilde{\theta})}{\partial \tilde{\theta}}$$

Q-Learning with Linear Function Approximation

Temporal Difference with Linear Function Approximation

$$\delta_k(\tilde{\theta}) = r(x_k, u_k) + \alpha \max_v \tilde{\theta}^T \phi(x_{k+1}, v) - \tilde{\theta}^T \phi(x_k, u_k)$$

Q-Learning with Linear Function Approximation

$$\tilde{\theta} \leftarrow \tilde{\theta} - \frac{1}{2} \beta_k \frac{\partial \delta_k^2(\tilde{\theta})}{\partial \tilde{\theta}}$$

Q-Learning with Linear Function Approximation

$$\tilde{\theta} \leftarrow \tilde{\theta} + \beta_k \delta_k(\tilde{\theta}) \phi(x_k, u_k)$$